

预备知识 P01: PyTorch 与张量

学大模型只用 HuggingFace 现成 API 把模型"跑起来"是远远不够的——一旦想读懂 Transformer 的 forward 在做什么、自己改一改 attention 的实现、写训练循环、调试形状报错，几乎每一步都要直接读写张量、在维度上做 reshape / transpose、靠 autograd 把梯度自动回传。这一章把这些最底层的工具讲清楚，给后续所有动手内容打底。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可✓。本章不加载任何大模型，PyTorch CPU 版就能跑通全部示例；有 GPU 当然更好——能顺手验证 `device` 相关的代码，但不是必须。

一、PyTorch 是大模型训练的基础底层框架

主流大模型（Qwen、LLaMA、DeepSeek、Mistral）以及训练 / 微调 / 推理生态（HuggingFace `transformers` / `peft` / `trl` / `accelerate`、vLLM、SGLang）几乎全部基于 PyTorch。读论文实现、跟着官方仓库 debug、自己写训练脚本，都绕不开两类操作：

- 张量（Tensor）操作：把数据组织成多维数组，做矩阵乘 / reshape / broadcasting。Transformer 的 forward 就是一长串张量变换。
- 自动求导（autograd）：训练就是「前向算 loss → 反向求 $\partial \text{loss} / \partial \theta$ → 优化器拿梯度更新参数」。手写偏导不现实，PyTorch 帮我们记下计算图，调一次 `loss.backward()` 就把全部梯度算好。

只要把张量四个属性（shape / dtype / device / requires_grad）和 autograd 这套机制摸熟，后续看 Attention、看 LoRA 的 `A @ B`、看训练循环里 `loss.backward()` 都不会卡住。预备知识阶段不追求把 PyTorch 学完——只学最常用的 30%，后面遇到新 API 再查文档。

二、张量（Tensor）：四个核心属性

张量（tensor）就是多维数组——0 维是标量、1 维是向量、2 维是矩阵、3 维及以上没有专门的数学名字。深度学习里 99% 的数据（图片、文本、激活值、权重、梯度）都装在张量里。

```
import torch
x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
print(x.shape)      # torch.Size([2, 3])
print(x.dtype)      # torch.float32
print(x.device)      # cpu
print(x.requires_grad) # False
```

每个 tensor 都有这四个身份证字段——后续 99% 的报错都来自其中之一不匹配 ("shape 对不上"、"dtype 不一致"、"一个在 cpu 一个在 cuda")。

2.1 shape：维度与形状

`shape` 是一个 tuple (元组, Python 里用圆括号包起来的不可变序列)，描述张量每一维的长度。维度数 (即 `len(shape)`) 叫 rank (秩) 或 ndim。

Rank	名字	例子	典型用途
0	标量 (scalar)	<code>torch.tensor(3.14)</code>	loss 的最终值
1	向量 (vector)	<code>torch.tensor([1, 2, 3])</code>	一个 token 的 embedding
2	矩阵 (matrix)	<code>(B, D)</code>	一个 batch 的特征向量
3	3 维张量	<code>(B, L, D)</code>	一个 batch 的 token embedding 序列 (batch、seq_len、hidden_dim)
4	4 维张量	<code>(B, H, L, D/H)</code>	multi-head attention 里把 hidden_dim 拆成多头之后的形状

LLM 里最常见的形状是 `(B, L, D)`：B = batch size、L = sequence length、D = hidden dim——后续 attention / FFN 等几乎所有模块都基于这种 3 维张量。

注意：`tensor.size()` 等价于 `tensor.shape` ——前者是方法调用、后者是属性访问，都返回一个 `torch.Size` (tuple 的子类)；想拿单维长度则 `tensor.size(dim)` 等价于 `tensor.shape[dim]`，两者都返回 `int`。

2.2 dtype：数值精度

`dtype` (data type) 决定每个元素占多少字节、能表示多大范围的数。

dtype	字节	能表达的范围	LLM 里的用途
<code>torch.float32</code> (<code>torch.float</code>)	4	大、精度高	默认；训练时主参数与梯度一般是 fp32
<code>torch.float16</code> (<code>torch.half</code>)	2	范围小、易上溢	T4 (Turing 架构) 半精度；推理常用

dtype	字节	能表达的范围	LLM 里的用途
<code>torch.bfloat16</code>	2	范围与 fp32 相同、精度低	A100/L4 起步的训练首选；不易上溢
<code>torch.int64</code> （ <code>torch.long</code> ）	8	整数	token id、index
<code>torch.bool</code>	1	True / False	mask（attention mask、causal mask）

T4 不原生支持 bf16——T4（Turing 架构）没有 bf16 硬件路径，强行用 bf16 会回退到软件模拟、速度反而更慢，因此在 T4 上加载半精度模型时要选 `torch.float16`。Ampere 及以上（A100 / L4 / RTX 30 系起）才能享受 bf16 的硬件加速。

dtype 不一致是新手最常见的报错来源。例：

```
a = torch.tensor([1, 2, 3])          # 默认 int64
b = torch.tensor([1.0, 2.0, 3.0])    # 默认 float32
a + b    # 这里 PyTorch 会自动把 a 提升到 float32 再相加 (type promotion)
a @ b    # ✖ 报 RuntimeError: dot : expected both vectors to have same dtype, but found Long and Float
```

注意 type promotion 只发生在逐元素运算（加减乘除、比较）上；`@` / `matmul` / `dot` 这类线性代数算子不做自动提升，两边 dtype 不一致直接报错——上面那条真实报错文本就是 int64 向量点乘 float32 向量的结果。

显式转换：`x.float()` / `x.long()` / `x.to(torch.bfloat16)`。

2.3 device：在 CPU 还是 GPU 上

`device` 标记张量存在哪儿。常见取值 `cpu` / `cuda:0` / `cuda:1`。两个张量必须在同一 device 才能做运算，否则会报：

```
RuntimeError: Expected all tensors to be on the same device, ...

x = torch.tensor([1.0, 2.0])          # cpu
y = torch.tensor([3.0, 4.0]).to("cuda") # cuda:0 (需有 GPU)
z = x + y                             # ✖ 报错: device 不一致
z = x.to(y.device) + y                 # ✔ 把 x 搬到同一 device 再加
```

搬运设备统一用 `.to(device)`，不推荐 `.cuda()`——前者更通用，同一份代码不改一行就能跑 cpu / cuda / mps（Apple 芯片）。

实际推理 / 训练代码里随处可见的 `inputs.to(model.device)` 就是这个意思——把输入张量搬到模型所在的设备（GPU），才能做 forward。

2.4 requires_grad: 是否参与 autograd

只有 `requires_grad=True` 的张量才会被 autograd 记录其计算历史，进而能反向求导。

- 模型参数（`nn.Parameter`，`nn.Linear` 等模块内部的 weight / bias）默认 `requires_grad=True`
- 输入数据（input_ids、图片像素）默认 `requires_grad=False` ——我们不需要对输入求导（除了"adversarial attack"（对抗攻击：通过对输入加微小扰动让模型误判）这类特殊场景）
- 推理时整体禁用：`with torch.no_grad():`，节省内存

具体怎么用，留到第 7 节展开。

三、创建张量的几种方式

```
import torch

# 1. 从 Python 列表 / 元组 / numpy 数组构造
torch.tensor([1, 2, 3])          # 推断 dtype = int64
torch.tensor([1.0, 2.0])        # float32

# 2. 给定形状, 元素全 0 / 全 1 / 未初始化
torch.zeros(2, 3)               # 2x3 全 0
torch.ones(2, 3)                # 2x3 全 1
torch.empty(2, 3)               # 2x3 未初始化 (值是垃圾内存, 不要直接用)

# 3. 等差序列
torch.arange(0, 10, 2)          # [0, 2, 4, 6, 8], 类比 Python range
torch.linspace(0.0, 1.0, 5)     # 在 [0, 1] 上等距取 5 个点

# 4. 随机张量 (深度学习里最常用)
torch.rand(2, 3)                # 每个元素独立采样自均匀分布 [0, 1)
torch.randn(2, 3)               # 每个元素独立采样自标准正态 N(0, 1)
torch.randint(0, 100, (2, 3))   # 每个元素独立采样自整数区间 [0, 100), 形状由最后一个参数指定

# 5. 与已有张量同 shape / dtype / device
x = torch.randn(2, 3, device="cpu")
torch.zeros_like(x)             # 和 x 完全同形同 dtype 同 device 的全 0 张量
```

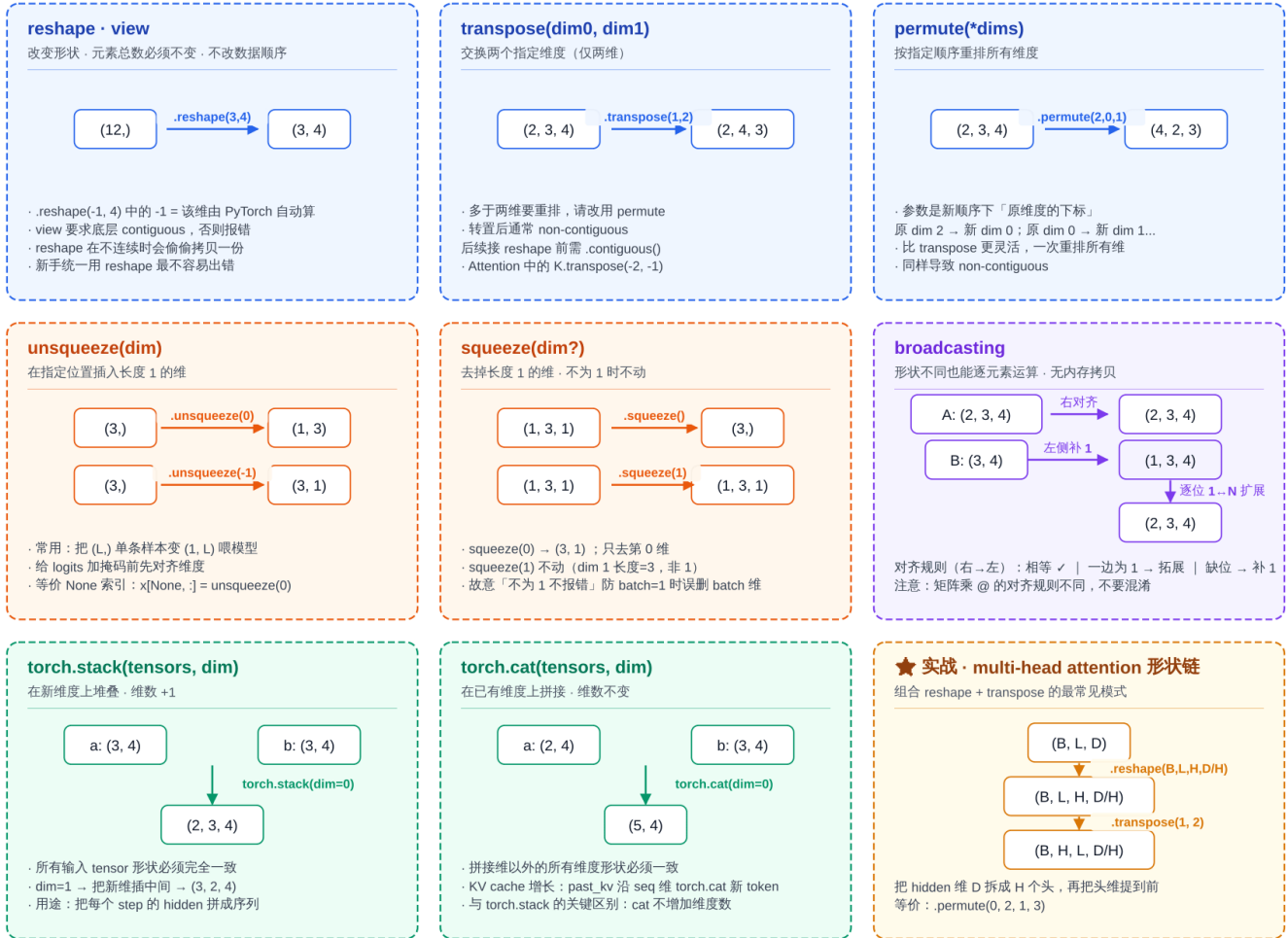
复现性提示：调用 `torch.manual_seed(42)` 后再产生随机张量，每次运行结果一致；写实验代码、对比同一份模型在不同超参下的表现时几乎都要先固定 seed。

四、形状操作：reshape / view / transpose / permute / squeeze

模型里张量形状变换非常密集，下面这几个 API 必须烂熟于心。本节及下节（broadcasting）涉及的所有形状变换，可以先看下面这张速查图——每个变换都标了 from-shape → to-shape，下文再逐项详细展开。

PyTorch 张量形状操作总览

每个变换标注 from-shape → to-shape · 颜色按操作语义分组 (蓝=重排数据 · 橙=增删 size-1 维 · 紫=Broadcasting · 绿=拼接多张量 · 黄=实战组合)



速记口诀: reshape 改观察方式 (数据不动) | transpose / permute 换轴 | squeeze / unsqueeze 加减 1 维 | broadcasting 自动对齐 | torch.stack 增维 / torch.cat 同维拼

```
import torch
```

```
x = torch.arange(12)
```

```
# shape (12,), 元素 [0, 1, ..., 11]
```

reshape(*shape) / view(*shape): 在不改动数据的前提下改变形状, 新形状元素总数必须等于原形状。

```
y = x.reshape(3, 4)
y = x.view(3, 4)
y = x.reshape(-1, 4)
```

```
# (12,) → (3, 4)
# 等价; 要求底层内存连续 (contiguous)
# -1 表示“剩下的维度自动算”, 等价 reshape(3, 4)
```

view vs reshape 的区别: **view** 要求底层内存连续, 否则报错; **reshape** 在不连续时会偷偷拷贝一份。新手统一用 **reshape** 最不容易出错; 只有在性能敏感的代码里 (避免拷贝), 且明确知道当前张量连续时再用 **view**。

transpose(dim0, dim1): 交换两个指定维度。

```
x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
x.transpose(0, 1)
# tensor([[1, 4],
```

```
# shape (2, 3)
# → shape (3, 2), 行列互换
```

```
#      [2, 5],
#      [3, 6]])

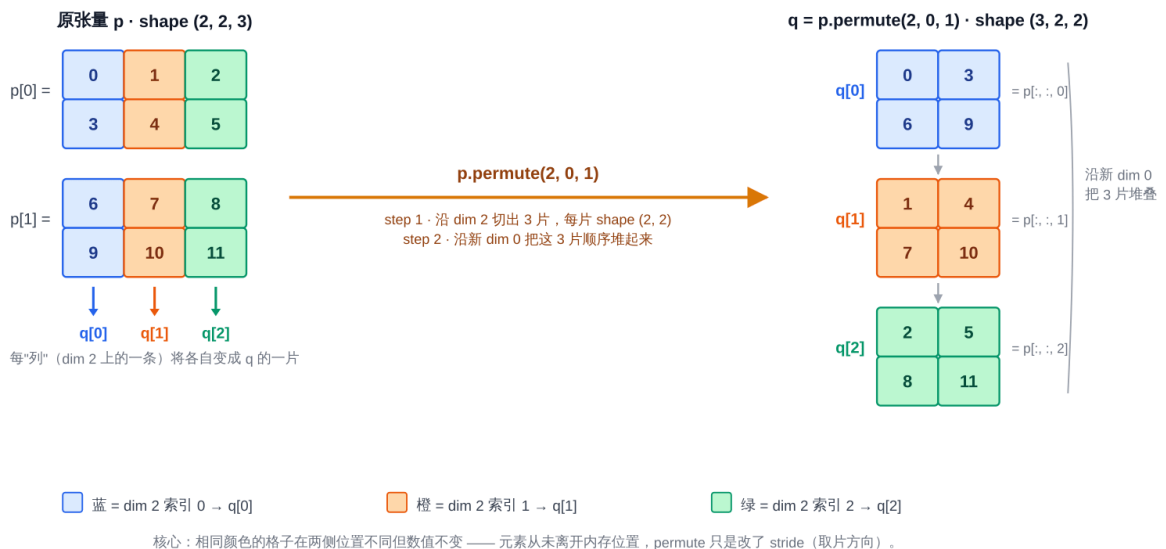
# 高维同理：只换两个指定维度
y = torch.randn(2, 3, 4)
y.transpose(1, 2).shape      # → (2, 4, 3)
```

permute(*dims)：按给定顺序重排所有维度，比 **transpose** 更灵活。

```
x = torch.arange(12).reshape(2, 2, 3) # shape (2, 2, 3)
# tensor([[[ 0,  1,  2],
#          [ 3,  4,  5]],
#         [[ 6,  7,  8],
#          [ 9, 10, 11]]])
x.permute(2, 0, 1)      # → shape (3, 2, 2)
# 含义：新 dim 0 = 原 dim 2, 新 dim 1 = 原 dim 0, 新 dim 2 = 原 dim 1
# 等价的索引关系：new[i, b, l] == old[b, l, i]
# tensor([[[ 0,  3],
#          [ 6,  9]],
#         [[ 1,  4],
#          [ 7, 10]],
#         [[ 2,  5],
#          [ 8, 11]]])
```

直观理解：「切薄片再堆叠」视角。上面那条 `new[i, b, l] == old[b, l, i]` 的索引等式不太好脑补，换成“切片+堆叠”的物理图像就直观得多——**permute** / **transpose** 都是「把张量按某一维切成薄片，再换一根轴把薄片堆起来」。

「切薄片再堆叠」视角：**permute(2, 0, 1)** 不搬数据，只换“取片方向”



图中用三种颜色标记 **dim 2** 的三个索引——可以看到相同颜色的格子在两侧位置不同但数值不变：左边按 **(b, l, d)** 视角“竖着”分布的同色列，到了右边变成 **(d, b, l)** 视角下的一片单色矩阵。元素本身一个没动。

以 `p.permute(2, 0, 1)` ($(2, 2, 3) \rightarrow (3, 2, 2)$) 为例:

1. 沿原 `dim 2` (长度 3) 把 `p` 切成 3 片, 每片 `shape (2, 2)`:

- `p[:, :, 0] = [[0, 3], [6, 9]]`
- `p[:, :, 1] = [[1, 4], [7, 10]]`
- `p[:, :, 2] = [[2, 5], [8, 11]]`

2. `permute(2, 0, 1)` 把"原 `dim 2`"提到新 `dim 0` —— 也就是把这 3 片沿新的第 0 维顺序堆起来。

3. 所以 `new[i]` 完全等于 `p[:, :, i]`, 三片对号入座, 没有任何元素的内存位置被改动——只是"取片方向"换了。

`transpose(d0, d1)` 是 `permute` 的特例, 同样可以这么理解。下文 multi-head attention 里反复出现的 $(B, L, H, D/H).transpose(1, 2) \rightarrow (B, H, L, D/H)$, 就是"沿 `H` 维切出 `H` 片, 每片 `shape (B, L, D/H)`, 再沿新的 `dim 1` 把这 `H` 片堆起来"——这样每个 head 各占一片, 后面就能按 head 并行做矩阵乘。

进阶补充: 逻辑上"切片再堆叠"似乎要重排数据, 但 PyTorch 实际只改了 `stride` (步长) ——底层内存里数据原封不动, 只是访问每个元素时按新 `stride` 跳着取。等到你对它调 `.contiguous()` (或调 `reshape` 而它无法用视图表达时), 那一刻才会真正发生一次数据拷贝; 而 `view` 永远不拷贝——遇到非连续张量它直接报错 (见上文 `reshape` vs `view` 的对比)。这也是 `transpose` / `permute` 后常需补一次 `.contiguous()` 的原因。

Transformer 的 multi-head attention 里典型的 $(B, L, D) \rightarrow (B, L, H, D/H) \rightarrow (B, H, L, D/H)$ 形状变换, 就是 `reshape` 之后再 `transpose(1, 2)` 或 `permute(0, 2, 1, 3)` ——这是后续读 attention 实现时会反复看到的模式。

squeeze / unsqueeze: 去掉 / 插入长度为 1 的维度。 `squeeze(dim)` 只有在 `dim` 这一维长度为 1 时才删, 否则原样返回 (不报错)。顺手提一句经典陷阱: 无参 `squeeze()` 会把所有长度 1 的维度一锅端——batch size 偶尔为 1 时连 batch 维一起删掉, 下游形状全乱 (PyTorch 官方文档专门警告过这一条); 防护手段就是显式传 `dim`, 只删你确定要删的那一维。

```
x = torch.randn(1, 3, 1)
x.squeeze().shape          # → (3, ), 去掉所有长度 1 的维度
x.squeeze(0).shape        # → (3, 1), 只去掉第 0 维
x.squeeze(1).shape        # → (1, 3, 1), dim 1 长度是 3, squeeze 不动它 (不报错)
x = torch.randn(3)
x.unsqueeze(0).shape       # → (1, 3), 在第 0 维插一个长度 1
x.unsqueeze(-1).shape      # → (3, 1), 在最后一维插一个长度 1
```

最常见的用途: 把 $(L,)$ 的单条样本变成 $(1, L)$ 的 batch, 喂给模型。

expand 与 **repeat**: 把长度 1 的维度"复制"扩展, 用于 broadcasting 之外的显式扩展。

- **expand**: 不拷贝内存, 只改张量的 `stride` (步长, 即"沿这一维走 1 步对应底层内存里跨多少个元素")——把 `stride` 设成 0, 多个逻辑位置就都指向同一块物理数据, 内存零开销。但 `expand` 只能扩展长度为 1

的维度，且绝不要对结果写入：它返回的不是只读视图——单点赋值（如 `t[0, 0] = 99`）会成功执行并把同一块物理数据连带原张量一起静默污染（最危险的路径，没有任何报错）；整块原地运算（如 `t += 1`）才会报 "more than one element ... refers to a single memory location"。

- `repeat`：真复制——分配新内存，把数据按指定次数物理拼接。能在任意维度上重复（不要求源维长度为 1），但内存代价是 `expand` 的 N 倍。

优先用 `expand`，只有需要写入结果、或源维长度不是 1 时才用 `repeat`。

```
import torch

x = torch.tensor([[1., 2., 3.]])      # shape (1, 3)
print(x.expand(4, 3))                # shape (4, 3), 4 行都是 [1, 2, 3]
# tensor([[1., 2., 3.],
#         [1., 2., 3.],
#         [1., 2., 3.],
#         [1., 2., 3.]])

print(x.expand(4, 3).stride())        # (0, 1) — 第 0 维 stride 为 0, 4 行共享同一块底层内存
print(x.repeat(4, 1))                # shape (4, 3), 输出一样但真复制了 4 份

# expand 只能扩展长度 1 的维度
y = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])      # shape (2, 3)
# y.expand(4, 3)                      # ✗ 报错: dim 0 长度是 2, 不是 1
print(y.repeat(2, 1))                # ✓ shape (4, 3): 把 y 整体在 dim 0 上重复 2 次
```

注意两者参数语义不同：`expand(*sizes)` 写目标形状（用 `-1` 表示该维不变）；`repeat(*times)` 写每一维重复多少次——容易看错参数。

`torch.stack` / `torch.cat`：把多个张量合并成一个，是上面图里另一组高频 API。注意这两个都是 `torch` 模块下的顶层函数（不是 `tensor` 方法），所以前面要带 `torch.`，与上面 `tensor.transpose(...)` / `tensor.permute(...)` 这种 `tensor` 方法不同。

- `torch.stack(tensors, dim)`：在新维度上堆叠，所有输入形状必须完全一致，输出维数 +1。
- `torch.cat(tensors, dim)`：在已有维度上拼接，除拼接维外其余维形状必须一致，输出维数不变。

```
a = torch.tensor([1., 2., 3.])      # shape (3,)
b = torch.tensor([4., 5., 6.])      # shape (3,)

torch.stack([a, b], dim=0)           # → shape (2, 3), 把两条向量"上下"摆成矩阵
# tensor([[1., 2., 3.],
#         [4., 5., 6.]])

torch.stack([a, b], dim=1)           # → shape (3, 2), 把两条向量"左右"插成列
# tensor([[1., 4.],
#         [2., 5.],
#         [3., 6.]])

torch.cat([a, b], dim=0)             # → shape (6,), 在已有维上首尾相接
# tensor([1., 2., 3., 4., 5., 6.])

# 高维只是同样规则的推广
```



```
torch.stack([torch.randn(3, 4), torch.randn(3, 4)], dim=0).shape # → (2, 3, 4)
torch.cat([torch.randn(2, 4), torch.randn(3, 4)], dim=0).shape # → (5, 4)
```

记忆要点：**torch.stack** 增加一维（要求形状完全一致）；**torch.cat** 在已有维上接长（其他维必须对齐）。
后续推理时维护 KV cache，每生成一个新 token 就要把它的 K/V 沿 sequence 维 **torch.cat** 到 **past_kv** 上。

五、Broadcasting：维度自动对齐

Broadcasting（广播）是 PyTorch（与 NumPy）的核心便利特性：形状不完全相同的两个张量，只要“兼容”就能直接做逐元素运算，PyTorch 会在底层把短的一边“虚拟地”扩展成长的形状（不真的复制内存）。

对齐规则：从最右侧（最后一维）开始，逐维比较两个 shape：

1. 维度长度相等 → 兼容
2. 一边是 1 → 兼容（这一维被扩展到另一边的长度）
3. 一边没有这一维（rank 更小）→ 兼容（左侧自动补 1）
4. 否则 → ✕ 报 `RuntimeError: The size of tensor a (3) must match the size of tensor b (5) at non-singleton dimension 1`（即 `(2, 3, 4) + (2, 5, 4)` 这种“既不相等、也没有一边是 1”的情形——报错里会点名是哪一维对不上）

例子：

```
A = torch.randn(2, 3, 4)           # shape (2, 3, 4)
B = torch.randn(1, 3, 4)           # shape (1, 3, 4), 左侧自动补成 (1, 3, 4)
A + B                               # 合法, 结果 (2, 3, 4)

A = torch.randn(2, 3, 4)           # shape (2, 3, 4)
B = torch.randn(2, 1, 4)           # 中间维是 1, 被广播到 3
A + B                               # 合法, 结果 (2, 3, 4)

A = torch.randn(2, 3, 4)           # shape (2, 3, 4)
B = torch.randn(2, 5, 4)           # 中间维不一致且都不是 1
A + B                               # ✕ 报错
```

LLM 里的典型场景：

- 给一个 batch 的所有样本加同一个 bias 向量：`(B, L, D) + (D,)` → bias 会被广播到 `(1, 1, D)` 再扩展
- 缩放每条样本的某一维：`(B, L, D) * (B, 1, 1)` → 第二个张量按样本提供独立的 scale

踩坑提醒：broadcasting 是把形状不同的两个张量“逐元素”运算变得可写，但不会把矩阵乘的不兼容形状救回来。
`(B, L, D) @ (K,)` ($K \neq D$) 会报错——矩阵乘要求相乘的内维对齐（这里末维 **D** 和 **K** 对不上），这是另一套规则（看下一节）。

六、张量运算：逐元素 vs 矩阵乘

PyTorch 的张量运算分两大类，初学最容易把它们搞混。

逐元素运算 (element-wise) : `+` `-` `*` `/` 以及 `torch.exp` `/` `log` `/` `sin` `/` `sigmoid` 等。两个张量 shape 必须相同 (或满足 broadcasting) ; 输出 shape 与输入相同。

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([10.0, 20.0, 30.0])
a + b          # tensor([11., 22., 33.])
a * b          # tensor([10., 40., 90.]) ← 是逐元素相乘, 不是点积!
torch.exp(a)   # tensor([2.7183, 7.3891, 20.0855])
```

* 不是矩阵乘——这是 PyTorch / NumPy 新手最容易栽的坑。

矩阵乘 / 点积: 用 `@` 或 `torch.matmul` 。规则:

- 两个 1 维张量: 点积 (dot product) , 输出标量。 $(D,) @ (D,) \rightarrow \text{scalar}$
- 一个 2 维 + 一个 1 维: 矩阵 $\times$ 向量。 $(M, D) @ (D,) \rightarrow (M,)$
- 两个 2 维: 矩阵乘。 $(M, K) @ (K, N) \rightarrow (M, N)$
- 高维: 把最后两维当矩阵乘, 前面的维度走 broadcasting。 $(B, M, K) @ (B, K, N) \rightarrow (B, M, N)$

```
A = torch.randn(2, 3)          # (2, 3)
B = torch.randn(3, 4)          # (3, 4)
A @ B                          # (2, 4), 矩阵乘

x = torch.randn(2, 3, 4)       # batch 矩阵
y = torch.randn(2, 4, 5)       # batch 矩阵
x @ y                          # (2, 3, 5), 每个 batch 独立做 (3, 4) @ (4, 5)
```

Attention 里 `Q @ K.transpose(-2, -1)` 就是上面这种 batch 矩阵乘。

常见数学函数: `x.sum(dim=...)` / `x.mean(dim=...)` / `x.max(dim=...)` / `torch.softmax(x, dim=...)` / `F.cross_entropy(...)` 等等——遇到了再查文档, 不必背。

七、Autograd: 自动求导引擎

PyTorch 的核心优势之一是 autograd: 自动记录张量上做过的所有运算, 并在调一次 `.backward()` 时把所有需要的偏导数沿着这张图算回来。没有 autograd, 深度学习训练就要手算每一层的偏导——这件事在 Transformer 这种几十层、参数过亿的网络里完全做不动。

7.1 计算图与反向传播

只要参与运算的张量里有任意一个 `requires_grad=True`，PyTorch 就会动态构建一张「计算图」（computational graph，简称 cgraph）：节点是张量，边是产生它的运算（`AddBackward` / `MulBackward` / ...）。

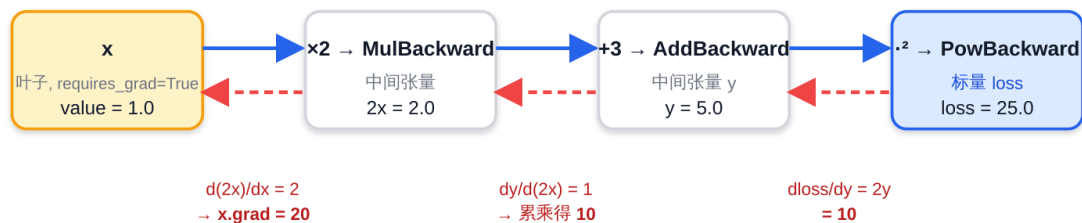
```
x (叶子节点, requires_grad=True)
|
▼
*2
|
▼
y = 2*x
|
▼
+3
|
▼
z = 2*x + 3
|
▼
pow(2)
|
▼
loss = (2*x + 3)^2    ← 标量, 可对 x 求偏导
```

调 `loss.backward()` 时，PyTorch 沿着这张图从 `loss` 反向走向每个 `requires_grad=True` 的叶子节点，按链式法则把偏导累加到叶子节点的 `.grad` 字段里。下面这张图把 forward 构建（蓝色箭头，左→右）与 backward 回传（红色虚线，右→左）画在一起：

autograd 计算图：forward 构建 + backward 沿链式法则回传

例：x=1.0, y = 2x + 3, loss = y²；手算 dloss/dx = 2y · 2 = 20

forward (构建计算图、保存中间值)



backward (链式法则反向累加梯度)

关键事实

- `loss.backward()` 要求 `loss` 是标量；非标量需要先 `.sum()` 或显式传 `gradient=` 参数
- 反向传播只把 `.grad` 累加到叶子节点；中间张量的 `.grad` 默认是 `None`（除非 `retain_grad()`）
- 计算图动态构建（define-by-run）：每次 forward 重新建图；推理用 `torch.no_grad()` 不建图，省一大半显存

叶子节点 (leaf tensor)：用户直接创建（不是由其他 tensor 运算得来）的 `requires_grad=True` 张量。模型参数都是叶子节点；中间结果不是。只有叶子节点的 `.grad` 会被填充——中间张量的 `.grad` 默认是 `None`。

计算图是动态的：每次 forward 都会重新构建（PyTorch 称为 `define-by-run`），这与早期 TensorFlow 1.x 的 `define-and-run`（先静态定义图、再喂数据）相反。动态图调试方便——你可以在 forward 中随便加 `print` 或条件分支，但代价是没有静态图的全局优化空间（`torch.compile` / `TorchScript` 是后来对此的补救）。

7.2 一个简单可手算的例子

下面用一个最小例子——用二次曲线 $\hat{y} = ax^2 + bx + c$ 拟合 3 个数据点：3 个参数 (a, b, c) 、3 个样本，整个训练步全程都能用纸笔手算到底，正好把后续 LLM 训练里“参数 $\rightarrow$ forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ 优化器更新”这条流水线完整跑一遍。

问题设定。假设真实曲线是 $y = x^2$ （事先并不知道，否则就用不着训练了），手里有 3 条样本： $(x_1, y_1) = (-1, 1)$ ， $(x_2, y_2) = (0, 0)$ ， $(x_3, y_3) = (1, 1)$ 。三个参数初值都设成 0（即“先猜 $\hat{y} \equiv 0$ ”）。

损失用平方误差之和：

$$L = \sum_{i=1}^3 (\hat{y}_i - y_i)^2$$

forward：所有 $\hat{y}_i = 0$ ，逐点误差 $\hat{y}_i - y_i$ 分别为 $-1, 0, -1$ ，所以 $L = 1 + 0 + 1 = 2$ 。

手算梯度（链式法则， $\hat{y}_i = ax_i^2 + bx_i + c$ ）：

$$\frac{\partial L}{\partial a} = \sum_i 2(\hat{y}_i - y_i) \cdot x_i^2, \quad \frac{\partial L}{\partial b} = \sum_i 2(\hat{y}_i - y_i) \cdot x_i, \quad \frac{\partial L}{\partial c} = \sum_i 2(\hat{y}_i - y_i)$$

代入 $\hat{y}_i - y_i = (-1, 0, -1)$ 与 $x_i = (-1, 0, 1)$ ：

- $\partial L / \partial a = 2 \cdot (-1) \cdot 1 + 0 + 2 \cdot (-1) \cdot 1 = -4$
- $\partial L / \partial b = 2 \cdot (-1) \cdot (-1) + 0 + 2 \cdot (-1) \cdot 1 = 0$
- $\partial L / \partial c = 2 \cdot (-1) + 0 + 2 \cdot (-1) = -4$

让 PyTorch 验证一遍：

```
import torch

# 3 个样本（固定，不需要梯度）
x_data = torch.tensor([-1.0, 0.0, 1.0])
y_data = torch.tensor([1.0, 0.0, 1.0])

# 3 个模型参数，初值全 0
a = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
c = torch.tensor(0.0, requires_grad=True)

# forward: 用 broadcasting 一次算出 3 个预测
y_hat = a * x_data ** 2 + b * x_data + c # shape (3,)
```

```

loss = ((y_hat - y_data) ** 2).sum()      # 标量

loss.backward()

print('loss =', loss.item())              # 2.0
print('a.grad =', a.grad.item())        # -4.0 ✓
print('b.grad =', b.grad.item())        # 0.0 ✓
print('c.grad =', c.grad.item())        # -4.0 ✓

```

梯度告诉我们什么？在当前 $(a, b, c) = (0, 0, 0)$ 这个点上：

- $\partial L / \partial a = -4 \rightarrow$ "a 增大一点，L 会减小" $\rightarrow$ 应该把 a 往正方向调
- $\partial L / \partial b = 0 \rightarrow$ "在当前点改 b 对 L 没影响" $\rightarrow$ b 暂时不动
- $\partial L / \partial c = -4 \rightarrow$ 同样应该把 c 往正方向调

把每个参数沿自己梯度的反方向走一小步——这就是梯度下降：

```

lr = 0.1
with torch.no_grad():                    # 参数更新本身不要进计算图
    a -= lr * a.grad                    # 0 - 0.1 * (-4) = 0.4
    b -= lr * b.grad                    # 0 - 0.1 * 0 = 0
    c -= lr * c.grad                    # 0 - 0.1 * (-4) = 0.4

# 用新参数再 forward 一次，看 loss 有没有降
y_hat = a * x_data ** 2 + b * x_data + c
loss = ((y_hat - y_data) ** 2).sum()

print('更新后 (a, b, c) =', (a.item(), b.item(), c.item())) # (0.4, 0.0, 0.4)
print('新 y_hat =', y_hat.tolist())                        # [0.8, 0.4, 0.8]
print('新 loss =', loss.item())                            # 0.24 — 从 2.0 大幅下降

```

仅一步更新，L 就从 2.0 降到 0.24。把这一套循环重复几千几万次，参数会逐步逼近真实值

$(a, b, c) = (1, 0, 0)$ ，L 最终收敛到 0（3 个参数刚好够精确拟合 3 个点）。这就是所有深度学习训练（包括 LLM 微调）的本质——区别只是参数从 3 个变成几十亿个、样本从 3 条变成一个 batch、loss 从平方误差换成交叉熵。**autograd** 的作用就是把"求每个参数的 **.grad**"这件事自动化——你只需要写出 forward 怎么算 loss，反向求导 PyTorch 全部代办。

小细节：注意 c 的最优值是 0，但更新一步后反而跑到 0.4。梯度只看当前点的瞬时下降方向，不保证一路指向全局最优——后续 a 增大、其他点的预测变好以后，c 会被拉回 0 附近。这是梯度下降的常态。

关键点：

- `loss.backward()` 要求 `loss` 是标量（0 维张量）。如果 `loss` 是向量/矩阵，需要先 `loss.sum().backward()` 或显式传 `gradient=` 参数指定上游梯度。实践中训练 `loss` 总是 `loss_fn(...).mean()` 这种标量——这条规则极少违反。这里用 `.sum()` 而不是 `.mean()` 只是为了让手算数字更干净，真实代码里 `.mean()` 更常见（梯度会按 $1/N$ 缩放）。
- 反向传播只更新叶子节点的 `.grad`。`y_hat` 是中间张量，`y_hat.grad` 是 `None`（除非显式调用 `y_hat.retain_grad()`）。

- 参数更新一定要包在 `torch.no_grad()` 里。不包会怎样？原地写法 `a -= lr * a.grad` 会当场报错——`RuntimeError: a leaf Variable that requires grad is being used in an in-place operation.`（autograd 禁止对正在追踪梯度的叶子张量做原地修改）；而非原地写法 `a = a - lr * a.grad` 不报错，但“更新”这一步会悄悄被计入计算图——`a` 从叶子变成中间张量，下一轮 `backward` 后 `a.grad` 变成 `None`，训练静默坏掉。包上 `no_grad()`，两种写法都安全（原地写法还能省一次内存分配）。

7.3 关闭梯度：`torch.no_grad` 与 `detach`

构建计算图本身要花内存（每个中间张量都得保留下来给反向用），推理时完全不需要——所以推理一定要在 `torch.no_grad()` 里跑：

```
with torch.no_grad():
    output = model(input_ids)    # 不构建计算图，省一大半显存、还稍快一些
```

所有推理 / 生成代码（如 `model.generate(...)`）都应当包在 `with torch.no_grad():` 里就是这个原因——能把推理时的显存占用砍掉一大半。

`detach()` 是针对单个张量的“切断”——返回一个新张量，和原张量共享数据但与计算图断开。常见用途：

- 保存 loss 数值用于打印 / 记录到日志：`losses.append(loss.detach().cpu().item())`——避免无意中拉住整张计算图，导致内存溢出
- RLHF / 蒸馏里把“reference model 的输出”当成静态目标，不让梯度回流：`ref_logits = ref_model(x).detach()`

```
x = torch.tensor(1.0, requires_grad=True)
y = (2 * x + 3) ** 2

z = y.detach()                # 与 y 共享数据，但不在计算图里
print(z.requires_grad)        # False
```

7.4 梯度不会自动清零：`zero_grad()`

PyTorch 默认 `.grad` 是累加的——多次 `backward()` 会把梯度加到一起，而不是覆盖。这在某些场景（梯度累积、二阶求导）里有用，但对最普通的训练循环来说就成了陷阱：

```
# ✗ 错误版本：忘记 zero_grad，梯度会一步步累加，训练发散
for batch in loader:
    loss = compute_loss(batch)
    loss.backward()
    optimizer.step()

# ✔ 正确版本：每个 batch 开头清零
for batch in loader:
    optimizer.zero_grad()    # 把所有参数的 .grad 置零
    loss = compute_loss(batch)
```



```
loss.backward()
optimizer.step()
```

`optimizer.zero_grad()` 等价于把所有它管理的参数的 `.grad` 置零；也可以写 `model.zero_grad()`。梯度累积训练（用于显存不够、把 batch 拆成多步累加）就是故意省略某几步的 `zero_grad`，让梯度自然累加 N 步后再 `step + zero_grad`。

下一章 P02 会把这条循环完整跑一遍。

八、关键概念回顾

概念	一句话定义	典型用途
张量四属性	shape / dtype / device / requires_grad	99% 的 PyTorch 报错都来自其中之一不匹配
broadcasting	形状不同的张量按规则自动对齐做逐元素运算	给 batch 加 bias、按位置缩放、掩码与 logits 相加等
@ vs *	@ 是矩阵乘 / 点积；* 是逐元素乘	几乎所有线性层、attention、低秩矩阵都用 @
计算图	forward 时动态构建、记录运算依赖	任何带训练的代码
.backward()	沿计算图回溯，把偏导累加到叶子节点 .grad	训练循环里把 loss 转化成参数梯度
torch.no_grad()	临时关闭计算图构建，推理省显存	推理 / 评估 / 生成代码统一这样写
optimizer.zero_grad()	训练循环开头清零梯度，避免累加	普通训练循环里的标配；故意累加多步可实现梯度累积

九、本章小结

这一章把 PyTorch 用到的最底层工具讲清楚了：

- 张量 = 多维数组 + 四个属性（shape / dtype / device / requires_grad）——99% 的 PyTorch 报错都来自这四项之一不匹配

- 形状变换 `reshape` / `transpose` / `permute` / `squeeze` 是模型 forward 里最常见的操作；
`broadcasting` 让形状不同也能逐元素运算；`@` 是矩阵乘，`*` 是逐元素乘
- `autograd` 是 PyTorch 的核心优势：forward 自动记录计算图，`backward()` 一步把所有梯度回传到参数；
推理用 `torch.no_grad()`，训练循环开头记得 `zero_grad()`

预告 P02：下一章把这些零件组装起来——用 `nn.Module` 写一个最小的 MLP，再走一遍「前向 → 反向 → 优化器」三步训练循环，在玩具数据上把网络真的训练到收敛。这套循环骨架在后续所有训练任务（语言模型预训练、SFT、LoRA、RLHF）里几乎一字不改。